i18n / Language Switcher

Problem: Build a language switcher that translates the UI and formats numbers, prices, dates and plurals per locale (often asked at Booking, Agoda).

Tech stack: React (Vite, JavaScript), no i18n library

Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task:

1. **Understand the requirements:** what it must do, what "good" means (performance, a11y), and what is out of scope. Ask before you assume.
2. **List the interactions:** every user action and the expected result. This becomes your build and test checklist.
3. **Sketch the architecture:** a few components/modules, each with one responsibility.
4. **Design the state:** the minimum state; derive the rest. Decide who owns each piece.
5. **Define data shapes and contracts:** sample data, API/function signatures.
6. **Trace one flow end to end:** input → state → side effect → output.
7. **Find the hard parts early:** solve them on paper before typing.
8. **Build in layers:** the simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases:** loading, error, empty, plus problem-specific ones.
10. **Wrap up:** what you handled, the trade-offs, what you would add with more time.

Time budget for 60 minutes

Phase	Minutes
Requirements + clarifying questions	5
Interactions, architecture, state, contracts	8
Core build (dictionaries, `t()`, provider, switcher)	22
Formatting (currency, date, plural, list), RTL, persistence	13
States, edge cases, polish	7
Wrap-up and trade-offs	5

Think out loud. Say each decision and why before you type it ("I keep the old language on screen until the new one has loaded, so there is no flash of keys"). The interviewer grades the reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A **language switcher** (dropdown) lists supported languages by their native names: English, हिन्दी, Français, العربية.
- Picking a language **re-renders all UI text** in that language without a page reload.
- `t(key, params)` translates a key with **placeholders** (`"Check-in: {date}"`) and **plurals** (`1 night` / `3 nights` , Arabic has 6 plural forms).
- **Locale-aware formatting:** numbers (`1,532` vs `1 532` vs `١٬٥٣٢`), currency, dates, relative time ("2 hours ago"), lists ("a, b and c").
- **RTL support:** Arabic flips the layout (`<html dir="rtl">`).
- **Persist** the choice; on first visit, **detect** it from the browser language.
- **Fallback:** a key missing in the current language shows the English text, never a blank.

Non-functional requirements

Area	What it means here
Performance	Load only the dictionary you need (lazy `import()`); cache loaded dictionaries; reuse `Intl` objects
Scalability	Adding a language = one dictionary file + one config line; no component changes
Error handling	A failed dictionary load keeps the current language and shows a retry; storage errors are ignored
Accessibility	Native `<select>` with a `<label>` ; `lang` on `<html>` and on each option so screen readers pronounce correctly; `role="status"` for loading
Usability	No flash of raw keys or empty text while switching; RTL layout mirrors correctly
Reusability	`useI18n()` hook works in any component; formatting is a separate pure module
Extensibility	Easy to swap the static `import()` for a fetch from a translation CDN later

Clarifying questions

1. Which languages, and is any **RTL** (Arabic/Hebrew)? → Yes, include Arabic.
2. Are translations **bundled** or fetched from a **server/CDN**? → Mock with lazy `import()` ; same contract as a fetch.
3. Is **currency tied to the language**, or does it come from the data? → From the data (a hotel prices in EUR). Locale only changes the *format*.
4. Do we need the **locale in the URL** (`/fr/hotels`) for SEO, or is client state enough? → Client state + `localStorage` for this round.
5. Should **content** (hotel names, reviews) be translated, or only UI chrome? → Only UI strings; content comes translated from the API.

6. Can I use a library like `react-i18next` / `FormatJS`? → No, build the core yourself; `Intl` APIs are allowed.

Assumptions

- Four locales: `en` (default + fallback), `hi`, `fr`, `ar` (RTL).
- Dictionaries are nested JS objects, loaded lazily, keyed by dot paths (`hotel.nights`).
- Plurals use the browser's `Intl.PluralRules` categories (`zero/one/two/few/many/other`).
- Price currency is a field of the data; we never convert currency.
- One demo screen: a hotel card that exercises every formatter.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
First visit, browser is `fr-CA`	Resolves to `fr` (base language match), UI in French
First visit, browser is `ja` (unsupported)	Falls back to `en`
Return visit with `app.locale = "hi"` saved	Opens in Hindi, ignores browser language
Pick "العربية" in the switcher	Shows "Loading…", old text stays; then UI switches to Arabic, `<html dir="rtl" lang="ar-EG">`
Switch fast: fr → ar → hi	Only Hindi wins; late fr/ar responses are ignored
Switch back to a language already loaded	Instant, no network
Press + / − on nights	Plural text updates (`1 night`, `2 nights`; Arabic `٣`, `ليلتان`, `ليلة واحدة`, `ليالٍ`) and total price recalculates
Key missing in Hindi	English text for that key; console warning in dev
Key missing everywhere	Key itself shown (`hotel.foo`) so it is visible in QA
Dictionary request fails	Current language stays; error message + Retry button
First-ever load fails	Plain error screen with Retry
`localStorage` blocked (private mode)	Works, just does not remember
Date `2026-12-24` viewed in New York	Shows Dec 24, not Dec 23

---

3. Design the architecture

Module	Responsibility
`locales.js`	Config: supported codes, native label, `dir`, `Intl` tag, lazy loader
`messages/*.js`	One dictionary per language
`translate.js`	Pure: lookup by path, interpolation, plural pick, fallback, locale resolution
`formatters.js`	Pure: cached `Intl` formatters (number, currency, date, relative, list)
`I18nProvider.jsx`	Owns locale state, loads/caches dictionaries, sets `<html lang dir>`, persists, exposes `t` + `fmt`
`useI18n.js`	Hook to read the context (throws outside the provider)
`LanguageSwitcher.jsx`	The dropdown + loading/error/retry UI
`HotelCard.jsx`	Demo consumer: uses `t` and `fmt`

```
main.jsx
└── I18nProvider            (state: requested, active, error; cache of dictionaries)
    └── App
        ├── LanguageSwitcher (reads requested/loading/error, calls setLocale)
        └── HotelCard        (reads t, fmt)

I18nProvider ──uses──► translate.js, formatters.js, locales.js ──lazy import──► messages/*.js
```

Start simple: a plain `t()` over one object first. The provider, lazy loading and formatters are split out only once each has a clear job.

---

4. Design the state / data model

State or field	Purpose
`requested`	The language the user picked (switcher shows this)
`active`	The language whose dictionary is loaded and on screen
`error`	Last load error, or `null`
`attempt`	Counter bumped by Retry to re-run the load
`cache` (ref)	`{ en: {...}, fr: {...} }` loaded dictionaries; not render state

```
const [requested, setRequested] = useState(initialLocale); // saved -> browser -> 'en'
const [active, setActive] = useState(null);
const [error, setError] = useState(null);
```

```
const [attempt, setAttempt] = useState(0);
const cache = useRef({});
```

Senior note.

- **Why two locale fields?** If we had one, switching would render the new language before its dictionary exists, so the UI flashes raw keys. `requested` drives the switcher; `active` drives the text. They differ only while loading.
- **Derived, not stored:** `loading = requested !== active && !error`, `dir`, the `Intl` tag, `t` and `fmt` are all computed from `active` in a `useMemo`.
- **Cache in a ref:** it only changes together with `setActive`, so it never needs its own re-render.
- **Ownership:** the provider owns all locale state. Components only read it through `useI18n()`. Nights count is local state of `HotelCard`, because nobody else needs it.

---

5. Design the data structures and contracts

Dictionary shape

```
// messages/en.js
export default {
  switcher: { label: 'Language', loading: 'Loading…', retry: 'Retry' },
  hotel: {
    checkIn: 'Check-in: {date}',
    nights: { one: '{count} night', other: '{count} nights' }, // plural object
  },
  amenity: { wifi: 'free Wi-Fi', pool: 'pool' },
};

// messages/ar.js  (Arabic needs all six categories)
nights: { zero: 'لا ليالٍ', one: 'ليلة واحدة', two: 'ليلتان',
          few: '{count} ليالٍ', many: '{count} ليلة', other: '{count} ليلة' }
```

Why nested objects + dot keys: they group by screen, are easy to split into files later, and a plural is just an object whose keys are `Intl.PluralRules` categories.

Locale config

```
export const LOCALES = {
  en: { label: 'English', dir: 'ltr', intl: 'en-US', load: () => import('./messages/en.js') },
  ar: { label: 'العربية', dir: 'rtl', intl: 'ar-EG', load: () => import('./messages/ar.js') },
};
```

The short code (`ar`) is our key; the `intl` tag (`ar-EG`) picks the region for formatting (digits, date order).

Mock API data (stable id)

```
// GET /api/hotels/htl_1042
{ id: 'htl_1042', name: 'Hôtel Lumière', city: 'Paris', rating: 8.7, reviews: 1532,
  checkIn: '2026-12-24', nights: 3,
  pricePerNight: { amount: 1249.5, currency: 'EUR' },
  amenities: ['wifi', 'pool', 'breakfast'], updatedHoursAgo: 2 }
```

Amenities are **codes**, not text, so the UI translates them (`amenity.wifi`). Money is `{ amount, currency }`, so the currency never depends on the UI language.

Hook contract

```
const {
  locale,      // 'fr' (active)
  requested,   // 'ar' while ar is loading
  dir,         // 'ltr' | 'rtl'
  loading, error,
  setLocale,   // (code) => void
  retry,       // () => void
  t,           // (key, params?) => string
  fmt,         // { number, currency, date, relative, list }
} = useI18n();

t('hotel.nights', { count: 3 })            // "3 nights" | "3 nuits" | "٣ ليالٍ"
fmt.currency(3748.5, 'EUR')                 // "€3,748.50" | "3 748,50 €" | "٣٬٧٤٨٫٥٠ €"
fmt.date(new Date(2026, 11, 24), { dateStyle: 'full' })
```

---

6. Trace the data flow

User picks **العربية** while the UI is in French:

```
<select onChange> value = "ar"
        |
        ▼
setRequested("ar")                          requested = ar, active = fr
        |                                   → switcher shows "Chargement…" (French text still)
        ▼
useEffect([requested]) runs
  missing = ["ar"]  (en already cached)
  LOCALES.ar.load()  → import('./messages/ar.js')
        |
        ▼  (promise resolves; effect not cancelled)
cache.ar = dict
```

```
setActive("ar")                          active = ar
localStorage.setItem("app.locale", "ar")
        │
        ▼
useEffect([active]) → <html lang="ar-EG" dir="rtl">
useMemo([active]) → new t / fmt for ar-EG
        │
        ▼
All useI18n() consumers re-render:
"٣,٧٤٨٫٥٠" ,"٣ ليالٍ" ,"٢٤ ديسمبر ٢٠٢٦" ,"تسجيل الوصول: الخميس، ٢٤ ديسمبر ٢٠٢٦ €"
layout mirrors through CSS logical properties
```

---

7. Important technical challenges

Challenge 1: Race condition when switching fast

Problem: The user picks French, then Arabic within 200 ms. The Arabic file is small and arrives first; the French file arrives later and overwrites it. The UI ends in French while the dropdown says Arabic.

Solution: Each load runs inside an effect keyed by `requested`. The effect's cleanup marks its own load as cancelled, so only the latest pick may call `setActive`. This also makes React StrictMode's double-run safe.

```
useEffect(() => {
  let cancelled = false;
  loadAll(requested).then(() => {
    if (!cancelled) setActive(requested);
  });
  return () => { cancelled = true; }; // a newer pick makes this one stale
}, [requested, attempt]);
```

Challenge 2: Plurals are not "1 vs many"

Problem: `count === 1 ? 'night' : 'nights'` breaks in most languages. French treats 0 as singular ("0 nuit"), Hindi too, Arabic has six forms (0, 1, 2, 3–10, 11–99, 100+), and Russian/Polish have their own rules.

Solution: Store a plural object per key with CLDR categories, and let `Intl.PluralRules` pick the category. Fall back to `other` if a category is missing. Number placeholders are formatted with the locale too, so Arabic shows `٣`.

```
if (msg && typeof msg === 'object' && typeof params.count === 'number') {
  msg = msg[plural.select(params.count)] ?? msg.other;
}
// new Intl.PluralRules('ar-EG').select(3)  -> "few"
// new Intl.PluralRules('fr-FR').select(0)  -> "one"
```

Challenge 3: Fallback chain and missing keys

Problem: A new feature ships with English text, but the Hindi translation is a week late. Without a fallback, the Hindi UI shows blank buttons or `undefined`.

Solution: Always load the default (`en`) dictionary next to the active one and look up in order `[active, en]`. If the key is missing in both, return the key itself (visible in QA) and warn in dev. Detection uses the same idea: `fr-CA` → `fr` → `en`.

```
for (const dict of [activeDict, enDict]) {
  const msg = pick(getByPath(dict, key));
  if (typeof msg === 'string') return interpolate(msg, params, formatNumber);
}
return key;
```

Challenge 4: Locale formatting (currency, dates, RTL)

Problem:

- Building `"€" + price.toFixed(2)` is wrong in French (`3 748,50 €`) and Arabic (Arabic digits, different separators).
- `new Date('2026-12-24')` is **UTC midnight**, so a user in New York sees **Dec 23**.
- Creating `new Intl.NumberFormat(...)` on every render of a 500-row list is slow.
- `margin-left` stays on the left in Arabic, so icons and spacing look wrong.

Solution: Use `Intl` for every format, cache formatter instances by `(type, locale, options)`, parse date-only strings as local dates, set `<html dir>`, and write CSS with logical properties (`margin-inline-start`, `padding-block`).

```
const cache = new Map();
export function getIntl(Ctor, tag, options = {}) {
  const key = `${Ctor.name}|${tag}|${JSON.stringify(options)}`;
  if (!cache.has(key)) cache.set(key, new Ctor(tag, options));
  return cache.get(key);
}
export const parseDateOnly = (iso) => {
  const [y, m, d] = iso.split('-').map(Number);
  return new Date(y, m - 1, d); // local midnight, not UTC
};
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Hard-code `en` dictionary + `t(key)` with `getByPath`	Static English page from keys

#	Step	Working result after the step
2	`interpolate` for `{placeholders}`	`Check-in: {date}` works
3	Add `hi` / `fr` / `ar` dictionaries + a context with `locale` state + `<select>`	Switching language works (all bundled)
4	Plurals with `Intl.PluralRules` + nights stepper	`1 night` / `٣ ليالٍ`
5	Fallback chain `[active, en]` → key	Missing keys never blank
6	`formatters.js`: number, currency, date, relative, list (cached)	Prices/dates per locale
7	`<html lang dir>` effect + logical CSS	Arabic mirrors correctly
8	Lazy `import()` per locale + `requested` vs `active` + cancel flag	No flash, no race
9	Detection (`navigator.languages`) + `localStorage` persistence	Remembers choice
10	Loading / error / retry states, boot screen	Failure is recoverable
11	a11y pass: label, `lang` on options, `role="status"`, focus ring	Screen-reader friendly
12	Node self-check of `translate.js` + `formatters.js`	Logic verified

9. Cover all states and edge cases

State / edge case	Handling
Initial (nothing loaded yet)	Provider renders a plain "Loading…" (no dictionary exists to translate it)
Switching (loading)	Old language stays on screen; switcher shows translated "Loading…" with `role="status"`
Success	`active = requested`, text + formats + `dir` update together
Error on switch	Stay on current language, show error + Retry (`attempt + 1` re-runs the effect)
Error on first load	Full error screen with Retry
Empty / missing key	Fallback to English, then the key itself; dev warning
Missing plural category	`?? msg.other`
Unknown placeholder	Left as `{name}` so the bug is visible
Unsupported browser language	Base-language match, then `en`
Corrupt saved value (`"xx"`)	`resolveLocale` skips it
`localStorage` throws	`try/catch`, still works
Rapid switching	Cancelled effects ignore late responses
StrictMode double effect	Same cancel flag; dictionary is cached so no double fetch result
Date-only strings and time zones	`parseDateOnly` builds a local date

State / edge case	Handling
RTL + mixed content (Latin hotel name in Arabic UI)	Browser bidi handles it; `<bdi>` for user-generated names if needed
0 nights	Plural `zero` / `other` text, Book button disabled
Large numbers (1,000,000 nights)	French `many` category: "1 000 000 de nuits"

10. Final structure

```
src/
├── main.jsx                  // wraps App in I18nProvider
├── App.jsx                   // header + switcher + demo card
├── App.css                   // logical properties for RTL
├── data/
│   └── hotel.js              // mock API response
├── components/
│   ├── LanguageSwitcher.jsx
│   └── HotelCard.jsx
└── i18n/
    ├── I18nContext.js
    ├── I18nProvider.jsx      // state, lazy load, cache, persistence, <html lang dir>
    ├── useI18n.js
    ├── locales.js            // config + lazy loaders
    ├── translate.js          // pure: lookup, interpolate, plural, fallback, resolveLocale
    ├── formatters.js         // pure: cached Intl formatters, parseDateOnly
    ├── translate.test.js     // node self-check
    └── messages/
        ├── en.js
        ├── hi.js
        ├── fr.js
        └── ar.js
```

11. Full working code

src/i18n/locales.js

```
// One place to add a language: its native name, text direction, Intl tag and a lazy loader.
export const LOCALES = {
  en: { label: 'English', dir: 'ltr', intl: 'en-US', load: () => import('./messages/en.js') },
  hi: { label: 'हिन्दी', dir: 'ltr', intl: 'hi-IN', load: () => import('./messages/hi.js') },
  fr: { label: 'Français', dir: 'ltr', intl: 'fr-FR', load: () => import('./messages/fr.js') },
  ar: { label: 'العربية', dir: 'rtl', intl: 'ar-EG', load: () => import('./messages/ar.js') },
```

```
};

export const SUPPORTED = Object.keys(LOCALES);
export const DEFAULT_LOCALE = 'en';
export const STORAGE_KEY = 'app.locale';
```

src/i18n/translate.js

```
// Pure helpers: no React, so they are easy to unit test.

// "hotel.nights" -> dict.hotel.nights
export const getByPath = (obj, path) =>
  path.split('.').reduce((node, key) => node?.[key], obj);

// "{count} nights" + { count: 3 } -> "3 nights". Unknown placeholders stay visible.
export const interpolate = (str, params = {}, formatNumber = String) =>
  str.replace(/\{(\w+)\}/g, (match, name) => {
    if (!(name in params)) return match;
    const value = params[name];
    return typeof value === 'number' ? formatNumber(value) : String(value);
  });

// Try each dictionary in order (active locale, then fallback). Missing everywhere -> the key itself.
export function translate({ dicts, plural, formatNumber }, key, params = {}) {
  for (const dict of dicts) {
    let msg = getByPath(dict, key);
    if (msg && typeof msg === 'object' && typeof params.count === 'number') {
      msg = msg[plural.select(params.count)] ?? msg.other;
    }
    if (typeof msg === 'string') return interpolate(msg, params, formatNumber);
  }
  if (import.meta.env?.DEV) console.warn(`[i18n] missing key "${key}"`);
  return key;
}

// ["fr-CA", "en-US"] -> "fr" if we support "fr". Exact match first, then the base language.
export function resolveLocale(candidates, supported, fallback) {
  for (const candidate of candidates) {
    if (!candidate) continue;
    const tag = candidate.toLowerCase();
    if (supported.includes(tag)) return tag;
    const base = tag.split('-')[0];
    if (supported.includes(base)) return base;
  }
  return fallback;
}
```

src/i18n/formatters.js

```
// Intl objects are expensive to build, so build each (type, locale, options) once.
const cache = new Map();

export function getIntl(Ctor, tag, options = {}) {
  const key = `${Ctor.name}|${tag}|${JSON.stringify(options)}`;
  if (!cache.has(key)) cache.set(key, new Ctor(tag, options));
  return cache.get(key);
}

// "2026-12-24" -> local midnight. new Date("2026-12-24") is UTC midnight and shows Dec 23 in the
Americas.
export const parseDateOnly = (iso) => {
  const [y, m, d] = iso.split('-').map(Number);
  return new Date(y, m - 1, d);
};

export const makeFormatters = (tag) => ({
  number: (n, options) => getIntl(Intl.NumberFormat, tag, options).format(n),
  currency: (amount, currency) =>
    getIntl(Intl.NumberFormat, tag, { style: 'currency', currency }).format(amount),
  date: (date, options = { dateStyle: 'medium' }) =>
    getIntl(Intl.DateTimeFormat, tag, options).format(date),
  relative: (value, unit) =>
    getIntl(Intl.RelativeTimeFormat, tag, { numeric: 'auto' }).format(value, unit),
  list: (items) => getIntl(Intl.ListFormat, tag, { type: 'conjunction' }).format(items),
});
```

src/i18n/I18nContext.js

```
import { createContext } from 'react';

export const I18nContext = createContext(null);
```

src/i18n/useI18n.js

```
import { useContext } from 'react';
import { I18nContext } from './I18nContext';

export function useI18n() {
  const ctx = useContext(I18nContext);
  if (!ctx) throw new Error('useI18n must be used inside <I18nProvider>');
```

```
  return ctx;
}
```

src/i18n/I18nProvider.jsx

```
import { useEffect, useMemo, useRef, useState } from 'react';
import { I18nContext } from './I18nContext';
import { LOCALES, SUPPORTED, DEFAULT_LOCALE, STORAGE_KEY } from './locales';
import { translate, resolveLocale } from './translate';
import { getIntl, makeFormatters } from './formatters';

function initialLocale() {
  let saved = null;
  try {
    saved = localStorage.getItem(STORAGE_KEY);
  } catch {
    // private mode / blocked storage: fall back to the browser language
  }
  return resolveLocale([saved, ...(navigator.languages ?? [navigator.language])], SUPPORTED,
DEFAULT_LOCALE);
}

export function I18nProvider({ children }) {
  const [requested, setRequested] = useState(initialLocale); // what the user picked
  const [active, setActive] = useState(null); // whose messages are on screen right now
  const [error, setError] = useState(null);
  const [attempt, setAttempt] = useState(0); // bump to retry a failed load
  const cache = useRef({}); // { en: {...}, fr: {...} } loaded dictionaries

  // Load the picked locale (+ the fallback) once, then swap. Old text stays until the new one is
ready.
  useEffect(() => {
    let cancelled = false; // a newer pick (or StrictMode re-run) makes this load stale
    setError(null);
    const missing = [...new Set([requested, DEFAULT_LOCALE])].filter((code) => !cache.current[code]);

    Promise.all(
      missing.map((code) =>
        LOCALES[code].load().then((mod) => {
          cache.current[code] = mod.default;
        }),
      ),
    )
      .then(() => {
        if (cancelled) return;
        setActive(requested);
```

```
      try {
        localStorage.setItem(STORAGE_KEY, requested);
      } catch {
        // not persisting is fine
      }
    })
    .catch((err) => {
      if (!cancelled) setError(err);
    });

  return () => {
    cancelled = true;
  };
}, [requested, attempt]);

// <html lang dir> drives screen-reader pronunciation, fonts, and RTL layout.
useEffect(() => {
  if (!active) return;
  document.documentElement.lang = LOCALES[active].intl;
  document.documentElement.dir = LOCALES[active].dir;
}, [active]);

const value = useMemo(() => {
  if (!active) return null;
  const tag = LOCALES[active].intl;
  const fmt = makeFormatters(tag);
  const ctx = {
    dicts: [cache.current[active], cache.current[DEFAULT_LOCALE]],
    plural: getIntl(Intl.PluralRules, tag),
    formatNumber: fmt.number,
  };
  return {
    locale: active,
    requested,
    dir: LOCALES[active].dir,
    loading: requested !== active && !error,
    error,
    setLocale: setRequested,
    retry: () => setAttempt((a) => a + 1),
    t: (key, params) => translate(ctx, key, params),
    fmt,
  };
}, [active, requested, error]);

// First load: nothing to show yet, and no dictionary to translate these two messages with.
if (!value) {
  return error ? (
```

```
      <p role="alert" className="boot">
        Could not load translations. <button onClick={() => setAttempt((a) => a + 1)}>Retry</button>
      </p>
    ) : (
      <p className="boot" aria-busy="true">Loading…</p>
    );
  }

  return <I18nContext.Provider value={value}>{children}</I18nContext.Provider>;
}
```

src/i18n/messages/en.js

```
export default {
  app: { title: 'Find your stay', subtitle: 'Prices, dates and numbers follow your language.' },
  switcher: { label: 'Language', loading: 'Loading…', error: 'Could not load this language.', retry:
'Retry' },
  hotel: {
    checkIn: 'Check-in: {date}',
    nights: { one: '{count} night', other: '{count} nights' },
    reviews: { one: '{count} review', other: '{count} reviews' },
    rating: 'Rated {score} / 10',
    includes: 'Includes {list}',
    total: 'Total price',
    updated: 'Price updated {time}',
    addNight: 'Add a night',
    removeNight: 'Remove a night',
    book: 'Book now',
  },
  amenity: { wifi: 'free Wi-Fi', pool: 'pool', breakfast: 'breakfast' },
};
```

src/i18n/messages/hi.js

```
export default {
  app: { title: 'अपना ठहराव खोजें', subtitle: 'कीमतें, तारीखें और संख्याएँ आपकी भाषा के अनुसार।' },
  switcher: { label: 'भाषा', loading: 'लोड हो रहा है…', error: 'यह भाषा लोड नहीं हो सकी।', retry: 'फिर से
कोशिश करें' },
  hotel: {
    checkIn: 'चेक-इन: {date}',
    nights: { one: '{count} रात', other: '{count} रातें' },
    reviews: { one: '{count} समीक्षा', other: '{count} समीक्षाएँ' },
    rating: 'रेटिंग {score} / 10',
    includes: 'शामिल: {list}',
    total: 'कुल कीमत',
```

```
    updated: 'कीमत {time} अपडेट हुई',
    addNight: 'एक रात जोड़ें',
    removeNight: 'एक रात हटाएँ',
    book: 'अभी बुक करें',
  },
  amenity: { wifi: 'मुफ़्त वाई-फ़ाई', pool: 'पूल', breakfast: 'नाश्ता' },
};
```

src/i18n/messages/fr.js

```
export default {
  app: { title: 'Trouvez votre séjour', subtitle: 'Les prix, dates et nombres suivent votre langue.' },
  switcher: { label: 'Langue', loading: 'Chargement…', error: 'Impossible de charger cette langue.',
retry: 'Réessayer' },
  hotel: {
    checkIn: 'Arrivée : {date}',
    nights: { one: '{count} nuit', many: '{count} de nuits', other: '{count} nuits' },
    reviews: { one: '{count} avis', other: '{count} avis' },
    rating: 'Noté {score} / 10',
    includes: 'Inclut {list}',
    total: 'Prix total',
    updated: 'Prix mis à jour {time}',
    addNight: 'Ajouter une nuit',
    removeNight: 'Retirer une nuit',
    book: 'Réserver',
  },
  amenity: { wifi: 'Wi-Fi gratuit', pool: 'piscine', breakfast: 'petit-déjeuner' },
};
```

src/i18n/messages/ar.js

```
export default {
  app: { title: 'ابحث عن إقامتك', subtitle: 'الأسعار والتواريخ والأرقام تتبع لغتك.' },
  switcher: { label: 'اللغة', loading: 'جارٍ التحميل…', error: 'تعذّر تحميل هذه اللغة.', retry: 'إعادة
المحاولة' },
  hotel: {
    checkIn: 'تسجيل الوصول: {date}',
    nights: {
      zero: 'لا ليالٍ',
      one: 'ليلة واحدة',
      two: 'ليلتان',
      few: '{count} ليالٍ',
      many: '{count} ليلة',
      other: '{count} ليلة',
    },
```

```
    reviews: {
      zero: 'لا توجد تقييمات',
      one: 'تقييم واحد',
      two: 'تقييمان',
      few: '{count} تقييمات',
      many: '{count} تقييمًا',
      other: '{count} تقييم',
    },
    rating: 'التقييم {score} / 10',
    includes: 'يشمل {list}',
    total: 'السعر الإجمالي',
    updated: 'تم تحديث السعر {time}',
    addNight: 'أضف ليلة',
    removeNight: 'احذف ليلة',
    book: 'احجز الآن',
  },
  amenity: { wifi: 'واي فاي مجاني', pool: 'مسبح', breakfast: 'فطور' },
};
```

src/components/LanguageSwitcher.jsx

```
import { useI18n } from '../i18n/useI18n';
import { LOCALES } from '../i18n/locales';

export default function LanguageSwitcher() {
  const { requested, setLocale, loading, error, retry, t } = useI18n();

  return (
    <div className="switcher">
      <label htmlFor="lang-select">{t('switcher.label')}</label>
      {/* Native select: keyboard, screen reader and mobile pickers for free */}
      <select
        id="lang-select"
        value={requested}
        onChange={(e) => setLocale(e.target.value)}
        aria-busy={loading}
      >
        {Object.entries(LOCALES).map(([code, { label, intl }]) => (
          <option key={code} value={code} lang={intl}>
            {label}
          </option>
        ))}
      </select>
      {loading && <span role="status">{t('switcher.loading')}</span>}
      {error && (
        <span role="alert" className="switcher-error">
```

```
          {t('switcher.error')} <button onClick={retry}>{t('switcher.retry')}</button>
        </span>
      )}
    </div>
  );
}
```

src/components/HotelCard.jsx

```
import { useState } from 'react';
import { useI18n } from '../i18n/useI18n';
import { parseDateOnly } from '../i18n/formatters';

export default function HotelCard({ hotel }) {
  const { t, fmt } = useI18n();
  const [nights, setNights] = useState(hotel.nights);

  // Hotel name/city are content from the API: shown as-is, not translated by the UI.
  return (
    <article className="card">
      <h2>{hotel.name}</h2>
      <p className="muted">
        {hotel.city} · {t('hotel.rating', { score: hotel.rating })} ·{' '}
        {t('hotel.reviews', { count: hotel.reviews })}
      </p>

      <p>{t('hotel.checkIn', { date: fmt.date(parseDateOnly(hotel.checkIn), { dateStyle: 'full' }) })}
</p>
      <p>{t('hotel.includes', { list: fmt.list(hotel.amenities.map((a) => t(`amenity.${a}`))) })}</p>

      <div className="nights">
        <button onClick={() => setNights((n) => Math.max(0, n - 1))} aria-label=
{t('hotel.removeNight')}>
          −
        </button>
        <output aria-live="polite">{t('hotel.nights', { count: nights })}</output>
        <button onClick={() => setNights((n) => n + 1)} aria-label={t('hotel.addNight')}>
          +
        </button>
      </div>

      <div className="price">
        <span className="muted">{t('hotel.total')}</span>
        {/* Currency comes from the data (EUR); only its formatting follows the locale */}
        <strong>{fmt.currency(hotel.pricePerNight.amount * nights, hotel.pricePerNight.currency)}
</strong>
```

```
      </div>
      <p className="muted small">
        {t('hotel.updated', { time: fmt.relative(-hotel.updatedHoursAgo, 'hour') })}
      </p>

      <button className="primary" disabled={nights === 0}>
        {t('hotel.book')}
      </button>
    </article>
  );
}
```

src/data/hotel.js

```
// Mock of GET /api/hotels/:id. Content (name, city) arrives in its own language from the server.
export const HOTEL = {
  id: 'htl_1042',
  name: 'Hôtel Lumière',
  city: 'Paris',
  rating: 8.7,
  reviews: 1532,
  checkIn: '2026-12-24', // date-only, no time zone on purpose
  nights: 3,
  pricePerNight: { amount: 1249.5, currency: 'EUR' },
  amenities: ['wifi', 'pool', 'breakfast'],
  updatedHoursAgo: 2,
};
```

src/App.jsx

```
import { useEffect } from 'react';
import { useI18n } from './i18n/useI18n';
import LanguageSwitcher from './components/LanguageSwitcher';
import HotelCard from './components/HotelCard';
import { HOTEL } from './data/hotel';
import './App.css';

export default function App() {
  const { t, locale } = useI18n();

  useEffect(() => {
    document.title = t('app.title');
  }, [t, locale]);

  return (
```

```
    <main className="app">
      <header className="app-header">
        <div>
          <h1>{t('app.title')}</h1>
          <p className="muted">{t('app.subtitle')}</p>
        </div>
        <LanguageSwitcher />
      </header>
      <HotelCard hotel={HOTEL} />
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { I18nProvider } from './i18n/I18nProvider';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <I18nProvider>
      <App />
    </I18nProvider>
  </StrictMode>,
);
```

src/App.css

```
/* Logical properties (inline-start/end) flip automatically when <html dir="rtl"> */
body {
  margin: 0;
  font-family: system-ui, 'Segoe UI', 'Nirmala UI', 'Noto Sans Arabic', sans-serif;
  background: #f4f5f7;
  color: #1f2328;
}
.app { max-width: 640px; margin: 0 auto; padding: 24px 16px; }
.app-header { display: flex; flex-wrap: wrap; gap: 16px; justify-content: space-between; align-items:
flex-start; }
.app-header h1 { margin: 0 0 4px; }
.muted { color: #57606a; }
.small { font-size: 0.85rem; }
.boot { padding: 24px; }
```

```
.switcher { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; }
.switcher select { padding: 6px 10px; border-radius: 6px; font: inherit; }
.switcher-error { color: #cf222e; }

.card { background: #fff; border-radius: 12px; padding: 20px; margin-block-start: 24px; box-shadow: 0 1px 3px #0002; }
.card h2 { margin: 0 0 4px; }
.nights { display: flex; align-items: center; gap: 12px; margin-block: 12px; }
.nights button { inline-size: 32px; block-size: 32px; border-radius: 50%; border: 1px solid #d0d7de; background: #fff; cursor: pointer; }
.price { display: flex; justify-content: space-between; align-items: baseline; border-block-start: 1px solid #eee; padding-block-start: 12px; }
.price strong { font-size: 1.4rem; }
.primary { margin-block-start: 12px; inline-size: 100%; padding: 10px; border: 0; border-radius: 8px; background: #0969da; color: #fff; font: inherit; cursor: pointer; }
.primary:disabled { opacity: 0.5; cursor: not-allowed; }
:focus-visible { outline: 2px solid #0969da; outline-offset: 2px; }
```

src/i18n/translate.test.js

Run with `node src/i18n/translate.test.js` (the project has `"type": "module"`).

```
// Run: node src/i18n/translate.test.js
import assert from 'node:assert/strict';
import { translate, resolveLocale, interpolate } from './translate.js';
import { makeFormatters, parseDateOnly } from './formatters.js';
import en from './messages/en.js';
import ar from './messages/ar.js';
import fr from './messages/fr.js';

const ctxFor = (dict, tag) => ({
  dicts: [dict, en],
  plural: new Intl.PluralRules(tag),
  formatNumber: makeFormatters(tag).number,
});

const EN = ctxFor(en, 'en-US');
assert.equal(translate(EN, 'hotel.nights', { count: 1 }), '1 night');
assert.equal(translate(EN, 'hotel.nights', { count: 1532 }), '1,532 nights');
assert.equal(translate(EN, 'nope.missing'), 'nope.missing');
assert.equal(interpolate('Hi {name}, {x}', { name: 'Asha' }), 'Hi Asha, {x}');

const AR = ctxFor(ar, 'ar-EG');
assert.equal(translate(AR, 'hotel.nights', { count: 2 }), 'ليلتان');
assert.equal(translate(AR, 'hotel.nights', { count: 3 }), '٣ ليالٍ'); // "few" + Arabic digits
assert.equal(translate(AR, 'hotel.nights', { count: 11 }), '١١ ليلة'); // "many"
```

```
const FR = ctxFor(fr, 'fr-FR');
assert.equal(translate(FR, 'hotel.nights', { count: 0 }), '0 nuit'); // French: 0 is "one"
// Fallback: a key missing in French resolves from English
assert.equal(translate({ ...FR, dicts: [{}, en] }, 'hotel.book'), 'Book now');

assert.equal(resolveLocale(['fr-CA', 'en-US'], ['en', 'fr'], 'en'), 'fr');
assert.equal(resolveLocale([null, 'de-DE', 'hi-IN'], ['en', 'hi'], 'en'), 'hi');
assert.equal(resolveLocale(['ja'], ['en'], 'en'), 'en');

assert.equal(parseDateOnly('2026-12-24').getDate(), 24);
assert.match(makeFormatters('en-US').currency(3748.5, 'EUR'), /€3,748\.50/);

console.log('i18n self-check passed');
```

---

12. Wrap-up

Edge cases handled

- Fast switching and StrictMode double effects (cancel flag).
- No flash of keys: old language stays until the new one is ready.
- Real plural rules for every language, including Arabic's six forms.
- Fallback to English, then to the key; unknown placeholders stay visible.
- Locale-correct numbers, currency, dates, relative time and lists; date-only strings never shift a day.
- RTL via `<html dir>` + logical CSS; `lang` set for screen readers and fonts.
- Detection from `navigator.languages`, persistence, and blocked storage.
- Load failure with retry, both on first load and on switch.

Trade-offs to mention

- **Own `t()` vs `react-i18next` / `FormatJS`:** own code is ~40 lines and shows understanding; a library brings ICU message syntax (`select`, nested plurals), rich-text `<Trans>`, namespaces and tooling. In production I would pick FormatJS or i18next.
- **Simple `{name}` syntax vs ICU MessageFormat:** simple and fast, but cannot express gender/select or several plurals in one sentence.
- **One context for everything:** every consumer re-renders on switch. That is fine because a language switch *should* re-render all text, and it is rare.
- **Client-side locale vs URL locale:** `localStorage` is quick to build, but `/fr/...` URLs are better for SEO, sharing and SSR.
- **Whole-language files:** simple; for big apps split dictionaries per route/namespace.
- **Currency from data, not from language:** correct for a booking site (a Paris hotel is priced in EUR), but a "show prices in my currency" feature needs a separate currency setting + conversion API.

What to add with more time

- Unit tests with Vitest + React Testing Library: switch, race, fallback, RTL attribute.
- Fetch dictionaries from a translation CDN with versioned URLs and `Cache-Control`; preload on hover over an option.
- Locale in the URL + `hreflang` links; SSR so the first paint is already translated.
- `<Trans>` component for sentences with links/bold inside.
- A script that diffs dictionaries against `en` to list missing keys in CI; TypeScript key types for autocomplete.
- Separate currency and time-zone settings in the user profile.

---

13. Counter questions the interviewer may ask

Requirements

Q1. What is the difference between i18n and l10n?

i18n (internationalization) is engineering the app so it *can* support many languages: keys instead of strings, `Intl` formatting, RTL-safe CSS. l10n (localization) is the actual work for one market: translations, local currency, images, legal text.

Q2. Why show language names in their own language ("Français", not "French")?

A user who cannot read the current language must still find their own. Native names (endonyms) solve that. We also set `lang` on each `<option>` so a screen reader pronounces it correctly.

Q3. Should the locale decide the currency?

No. A French user booking in Tokyo pays in JPY or picks a currency. Locale decides *how* money is written (`3 748,50 €`), the data or a separate user setting decides *which* currency. Mixing them is a classic Booking/Agoda bug.

State design

Q4. Why keep both `requested` and `active` instead of one `locale`?

With one field, the UI re-renders in the new language before its dictionary exists, so it flashes keys or English. Two fields let the switcher reflect the choice immediately while the text changes only when it is ready.

Q5. Why is the dictionary cache a `useRef` and not state?

It is only updated right before `setActive`, which already triggers the render. Putting it in state would cause an extra render and make the effect depend on it.

Q6. Why Context and not Redux/Zustand?

Locale is low-frequency global state read by many components: exactly what Context is good at. A store adds nothing here. If the app already has a store, the locale can live there and the provider just reads it.

Challenge: race conditions

Q7. What if the user switches languages very fast on a slow network?

Each switch starts a new effect run; the cleanup of the old run sets `cancelled = true`, so a late response cannot call `setActive`. The dictionary still lands in the cache, so switching back later is instant.

Q8. Why not use `AbortController`?

With `fetch` you should, to stop wasted downloads. A dynamic `import()` cannot be aborted, and the cancel flag is still needed to ignore results. With a real API I would use both.

Q9. StrictMode runs the effect twice in dev. Does it break?

No. The first run is cleaned up (`cancelled = true`), the second run does the work. Loading is idempotent and cached, so nothing doubles.

Challenge: plurals and messages

Q10. Why not `count === 1 ? one : other`?

It is wrong for most languages: French and Hindi treat 0 as singular, Arabic has six forms, Polish chooses by the last digits. `Intl.PluralRules` encodes the CLDR rules for every locale, so the dictionary just provides the forms.

Q11. How would you handle gender or a sentence with two plurals?

Our `{name}` syntax cannot. I would switch to ICU MessageFormat (`{gender, select, female {…} other {…}}`, nested `plural`) via FormatJS's `IntlMessageFormat`, keeping the same `t(key, params)` API so components do not change.

Q12. How do you translate a sentence that contains a link, like "Read our terms"?

Never split it into three keys, because word order differs per language. Use one message with tags and a `<Trans>` component that maps tags to React elements: `t('legal', { a: (text) => <a href="/terms">{text}</a> })`.

Challenge: formatting and RTL

Q13. Why cache `Intl.NumberFormat` instances?

Constructing one parses locale data and is far slower than calling `.format()`. In a 500-row price list that is 500 constructions per render. Caching by `(type, locale, options)` makes each one a single `Map` lookup.

Q14. Why does `new Date('2026-12-24')` show Dec 23 for some users?

ISO date-only strings parse as UTC midnight; in UTC−5 that is Dec 23, 7 pm local. For check-in dates (no time), parse into a local date or format with `timeZone: 'UTC'`. For real timestamps, store UTC and format in the user's or hotel's time zone.

Q15. How do you make the layout work in RTL?

Set `<html dir="rtl">`, then use CSS logical properties (`margin-inline-start`, `inset-inline-end`, `text-align: start`) and flexbox, which flip automatically. Mirror directional icons (arrows) with `[dir=rtl] .icon { transform: scaleX(-1) }`, but not logos or media controls.

Q16. A Latin hotel name inside an Arabic sentence shows punctuation in the wrong place. Fix?

That is the Unicode bidi algorithm. Wrap user or API content in `<bdi>` (or use `dir="auto"`) so it is isolated from the surrounding direction.

Implementation and performance

Q17. What if we support 40 languages with 5,000 keys each?

Never bundle them all: lazy-load per locale (done) and split per namespace/route (`checkout.json`, `search.json`) so a page loads only its keys. Serve from a CDN with content-hashed URLs and long caching.

Q18. What if the translation API is slow on first load?

Inline the default language (or the detected one) in the HTML/SSR response so first paint is translated, then lazy-load others. Preload a language when the user opens the dropdown or hovers an option.

Q19. How do you avoid a flash of the wrong language on first paint?

Detect on the server from the URL or `Accept-Language` / a cookie, render that language with SSR, and set `<html lang dir>` in the HTML. Client-only apps can show a neutral loader until the dictionary arrives, as we do.

Q20. Every component re-renders on a language switch. Is that a problem?

No: a switch is rare and all visible text changes anyway. The concern would be a context value that changes for other reasons; ours is memoized on `[active, requested, error]`, so it only changes on switches.

Trade-offs and "why not X?"

Q21. Why not `react-i18next`?

In an interview the point is to show I understand lookup, plurals, fallback and loading. In production I would use i18next or FormatJS for ICU syntax, `<Trans>`, namespaces, extraction tools and translator workflows.

Q22. Why not store the locale only in the URL?

URLs are better for SEO (`hreflang`), sharing and SSR, and I would add them for public pages. For this round, `localStorage` plus browser detection is enough and the provider API would not change: `setLocale` would just call the router.

Q23. How do you catch missing translations before production?

A CI script flattens every dictionary and diffs its keys against `en`; missing keys fail the build or open a ticket for translators. In dev, `t()` warns in the console, and TypeScript can type `t()` keys from the `en` file for

autocomplete.

Q24. How would you test this?

Unit-test `translate` / `resolveLocale` /formatters in Node (done). With React Testing Library: render, change the select, `await findByText('Réserver')`, assert `document.documentElement.dir === 'rtl'` for Arabic, and mock a slow + fast loader to prove the race fix.